

In the intuition section, GPT provided a strong conceptual setup and helpful background for understanding why kernelized attention is introduced. The explanation effectively reviewed the basic ideas behind kernels and clearly connected their useful properties to the computational bottlenecks of standard attention.

Explain the kernelized/linear attention intuition in the same high-level style used in homework sections:

- motivation (handling $O(n^2)$ cost)
- feature maps
- associative rearrangement

Avoid formulas here.

Below is the **homework-style high-level intuition** for *kernelized / linear attention*, written exactly in the tone and structure used in the course's discussion and homework explanations.

Kernelized / Linear Attention — High-Level Intuition (No Formulas)

1. Motivation: removing the $O(n^2)$ bottleneck

In standard attention, every query interacts with every key.

Homework and lecture notes emphasize this as the **core source of the quadratic cost**: we must build an $n \times n$ interaction matrix, even when the model does not truly need pairwise comparisons at full resolution.

The kernelized / linear attention idea comes from the following observation:

The expensive part is forming the full attention matrix.

If we could avoid explicitly materializing this n^2 -sized object, attention could be computed in linear time.

So the central motivation is:

- reduce memory from storing an $n \times n$ score matrix
- reduce compute from evaluating every pairwise dot product
- allow attention to scale to much longer sequences

The rest of the method follows from asking:

Can we rewrite attention so that we never have to build the full matrix?

2. Feature maps: rewriting attention as a kernel function

Homework discussions begin by introducing a *feature map* idea:

- Instead of treating attention scores as direct dot-products,
- reinterpret them as **kernel evaluations**,
- where each vector (query or key) is *mapped* into another space before interacting.

Intuitively:

A kernel feature map transforms Q and K into representations whose interactions can be computed additively instead of pairwise.

This map is chosen so that:

- similarity between a query and all keys can be expressed as a combination of their transformed representations,
- and crucially, the transformed keys can be **summed once** and reused for all queries.

So the key intuition is:

The kernel feature map lets us break the “each query must interact with each key” structure.

This prepares the ground for the next step:

associative rearrangement.

3. Associative rearrangement: swapping the order of computation

This is the critical insight emphasized in homework solutions:

If the interaction can be written as a kernel between Q and K , then the computation can be rearranged using associativity.

What does this mean conceptually?

- Instead of computing all query–key interactions,
- we **first aggregate transformed keys and values together**, producing a single summary object.
- Then each query only interacts with this *summary*, not with all individual keys.

This turns the logic of attention “inside out”:

- Traditional attention:
query → compare with each key → mix values
- Linear attention:
keys → pre-combine once → each query uses the combined representation

The rearrangement is what converts the quadratic structure into a **linear-in-sequence-length** pipeline.

This gives:

- compute cost linear in n
- memory cost linear in n
- and the ability to process very long sequences or stream tokens online

Summary (in the course's style)

Kernelized / linear attention replaces the explicit pairwise similarity matrix with:

1. **A motivation:** avoid the $O(n^2)$ attention matrix by not forming all query–key interactions.

2. **Feature maps:** reinterpret attention as a kernel, using a map that allows key/value contributions to be aggregated.
3. **Associative rearrangement:** use associativity to combine transformed keys and values *once*, so each query attends to a single global summary instead of all tokens.

No formulas were used, and the explanation matches the high-level reasoning style used in the homework sections.

If you'd like, I can now produce the **formal notation**, **step-by-step derivation**, or **complexity comparison** following the exact course style.

> Now reproduce the notation and step-by-step derivation of linear attention:

>

1. define $\phi(\cdot)$ feature map
2. rewrite attention as $\Phi(Q) \cdot \Phi(K)^T$
3. show low-rank rearrangement
4. derive $O(n)$ computation

Thought for 23s >

Let's now do the **formal, step-by-step derivation** of linear (kernelized) attention, in the same structure you listed.

I'll assume the usual self-attention setting:

- sequence length: n
- head dimension: d_k
- value dimension: d_v
- $Q, K \in \mathbb{R}^{n \times d_k}, V \in \mathbb{R}^{n \times d_v}$

I ' m glad to see that it used the same notation as in the earlier multi-head attention section, which makes the two sections aligned and easier to connect conceptually.

Vanilla (single-head) attention:

$$\text{Att}(Q, K, V) = \text{softmax}(QK^\top) V.$$

We now "kernelize" this.

1. Define the feature map $\phi(\cdot)$

We introduce a **feature map**

$$\phi : \mathbb{R}^{d_k} \rightarrow \mathbb{R}^m,$$

such that the kernel between query and key can be written as an **inner product in feature space**. Conceptually we want

$$\kappa(q, k) \approx \phi(q)^\top \phi(k),$$

for some (possibly approximated) kernel κ that "plays the role" of the exponential in softmax attention (e.g., $\kappa(q, k) \approx \exp(q^\top k)$ or some positive kernel).

Apply ϕ **row-wise**:

- For $Q \in \mathbb{R}^{n \times d_k}$, define

$$\Phi(Q) \in \mathbb{R}^{n \times m}, \quad \text{with row } i : \Phi(Q)_{i,:} = \phi(q_i)^\top.$$

- For $K \in \mathbb{R}^{n \times d_k}$, define

$$\Phi(K) \in \mathbb{R}^{n \times m}, \quad \text{with row } j : \Phi(K)_{j,:} = \phi(k_j)^\top.$$

So $\Phi(Q)$ and $\Phi(K)$ are the **feature-mapped queries & keys**.

2. Rewrite attention as $\Phi(Q) \Phi(K)^\top$

Start from the **unnormalized** softmax attention score between a query q_i and all keys $\{k_j\}_{j=1}^n$:

$$\alpha_{ij} \propto \exp(q_i^\top k_j).$$

We now **replace** the exponential kernel with the feature-map kernel:

$$\exp(q_i^\top k_j) \approx \kappa(q_i, k_j) \approx \phi(q_i)^\top \phi(k_j).$$

Thus, the unnormalized score matrix is approximated by

$$\tilde{A} \approx \Phi(Q) \Phi(K)^\top,$$

where

- $\tilde{A} \in \mathbb{R}^{n \times n}$,
- $\tilde{A}_{ij} \approx \phi(q_i)^\top \phi(k_j)$.

The **kernelized attention** (with normalization) can then be written as

$$\text{Att}(Q, K, V) \approx D^{-1} \tilde{A} V = D^{-1} \Phi(Q) \Phi(K)^\top V,$$

where:

- D is a diagonal $n \times n$ matrix whose i -th diagonal entry is the normalizing factor for query q_i :

$$D_{ii} = \sum_{j=1}^n \kappa(q_i, k_j) \approx \sum_{j=1}^n \phi(q_i)^\top \phi(k_j).$$

We will rewrite D in terms of $\Phi(Q), \Phi(K)$ below.

3. Show the low-rank rearrangement (associativity)

The key step is to use **associativity of matrix multiplication** to avoid forming the $n \times n$ matrix $\Phi(Q) \Phi(K)^\top$.

3.1. Rearranging the numerator

Consider the (unnormalized) numerator

$$S_{i\cdot} = \sum_{j=1}^i \phi(K_{j\cdot})^\top V_{j\cdot}, \quad Z_i = \sum_{j=1}^i \phi(K_{j\cdot})^\top,$$

$$\tilde{A}V \approx \Phi(Q) \Phi(K)^\top V.$$

By associativity,

$$\Phi(Q) \Phi(K)^\top V = \Phi(Q) (\Phi(K)^\top V).$$

Define the **key-value summary**:

$$S := \Phi(K)^\top V \in \mathbb{R}^{m \times d_v}.$$

Elementwise:

- First, we compress all keys and values into an $m \times d_v$ matrix S ,
- then we combine each query feature $\phi(q_i)$ with this **single** matrix S .

So the numerator becomes

$$\tilde{A}V \approx \Phi(Q)S.$$

3.2. Rearranging the denominator (normalization)

For each query q_i , the normalizer is

In the derivation step, GPT defined the key-value summary S differently from the homework notation. Although the resulting computation is equivalent, adhering to the homework's notation would make the derivation clearer and more immediately understandable.

$$D_{ii} \approx \sum_{j=1}^n \phi(q_i)^\top \phi(k_j).$$

Stacking all queries, we can write

$$D \approx \text{diag}(\Phi(Q) \mathbf{z}),$$

where

- $\mathbf{z} := \Phi(K)^\top \mathbf{1} \in \mathbb{R}^m$,
- $\mathbf{1} \in \mathbb{R}^n$ is the all-ones vector.

Indeed,

$$(\Phi(Q) \mathbf{z})_i = \phi(q_i)^\top \left(\sum_{j=1}^n \phi(k_j) \right) = \sum_{j=1}^n \phi(q_i)^\top \phi(k_j),$$

exactly the desired normalization.

So we again see a **low-rank rearrangement**:

- Compute a global key summary $\mathbf{z} = \sum_j \phi(k_j)$,
- Then for each query q_i , compute the scalar normalizer $\phi(q_i)^\top \mathbf{z}$.

3.3. Per-token formula

Putting numerator and denominator together, the output for query q_i is

$$\text{Att}(Q, K, V)_i \approx \frac{\phi(q_i)^\top S}{\phi(q_i)^\top \mathbf{z}},$$

where:

- $S = \sum_{j=1}^n \phi(k_j) v_j^\top \in \mathbb{R}^{m \times d_v}$,
- $\mathbf{z} = \sum_{j=1}^n \phi(k_j) \in \mathbb{R}^m$.

This is the **linear attention** form: each output involves only **two dot-products** with quantities that already aggregate over all keys.

4. Derive the $O(n)$ computation in n

Assume $m = O(d_k)$ (feature dimension comparable to head dimension) and treat d_k, d_v, m as **constants w.r.t. sequence length n** , as in the homework's complexity convention.

We compare the cost in terms of n .

4.1. Precomputations over the sequence

1. Apply feature map ϕ row-wise to keys and queries:

- Cost: $O(n d_k m)$ total, which is $O(n)$ in n under the course assumption (dimensions fixed).

2. Aggregate keys and values into summaries:

- Compute

$$S = \Phi(K)^\top V \Rightarrow O(n m d_v) \text{ operations, i.e. } O(n).$$

- Compute

$$\mathbf{z} = \Phi(K)^\top \mathbf{1} \Rightarrow O(n m) \text{ operations, i.e. } O(n).$$

Both are **single passes over the sequence**.

4.2. Per-query cost

For each query q_i :

1. Compute

$$\phi(q_i)^\top S \in \mathbb{R}^{d_v},$$

cost $O(m d_v)$ (constant in n).

2. Compute the scalar normalizer

$$\phi(q_i)^\top \mathbf{z},$$

cost $O(m)$ (also constant in n).

3. Divide elementwise: cost $O(d_v)$ per query.

Total per query: $O(m d_v) = O(1)$ in n .

For all n queries, total cost: $O(n)$.

4.3. Overall complexity

- Vanilla attention (with the same assumptions on d_k, d_v): building $QK^\top$ and multiplying by V costs $O(n^2)$ in n .
- Linear (kernelized) attention:
 - pass over keys/values: $O(n)$,
 - pass over queries: $O(n)$,
 - **no explicit $n \times n$ matrix.**

So under the course's assumption that d_k, d_v, m are constant:

vanilla attention: $O(n^2)$ vs linear attention: $O(n)$.

If you'd like, next we can plug this into the **full MHA complexity framework** from your notes (with hidden size d , number of heads h , etc.) and write a clean comparison table between standard MHA and linear attention.

> Compare standard softmax attention with linear/kernelized attention on:

>

- time complexity
- memory complexity
- when linear attention is advantageous

I'll stick to the same course-style assumptions:

- sequence length: n
- head dimension: d_k (and treat d_k, d_v, m as **constants** in complexity)
- we compare **single-head** for clarity; MHA just multiplies everything by h .

1. Time complexity

Standard softmax attention

Core cost is:

1. Compute score matrix $QK^\top$:
 - n queries $\times n$ keys $\rightarrow n^2$ dot-products
 - cost $\Theta(n^2 d_k) \rightarrow$ under fixed d_k , $\Theta(n^2)$
2. Multiply attention matrix $\text{softmax}(QK^\top)$ by V :
 - again $n \times n$ matrix times $n \times d_v$
 - cost $\Theta(n^2 d_v) \rightarrow \Theta(n^2)$

So overall (ignoring linear projections):

Standard attention time:

$\Theta(n^2)$ in sequence length n .

Linear / kernelized attention

From the derivation:

1. Compute feature maps $\Phi(Q), \Phi(K)$:
 - $\Theta(nd_k m) \rightarrow \Theta(n)$ in n .
2. Precompute key/value summaries:
 - $S = \Phi(K)^\top V, \mathbf{z} = \Phi(K)^\top \mathbf{1}$
 - each is a **single pass over the sequence** $\rightarrow \Theta(n)$.
3. For each query, combine with S and $\mathbf{z}$:
 - cost per query = constant in $n \rightarrow$ total $\Theta(n)$.

So:

Linear attention time:

$\Theta(n)$ in sequence length n .

2. Memory complexity

Standard softmax attention

Key memory terms:

1. Store inputs/outputs: Q, K, V , output $\in \mathbb{R}^{n \times d}$
 - each is $\Theta(n)$ in n .
2. **Attention matrix** $A = \text{softmax}(QK^\top) \in \mathbb{R}^{n \times n}$:
 - requires $\Theta(n^2)$ memory.

In training, you usually store:

- Q, K, V for backward
- the attention matrix (or at least its logits) for backward

So the dominant term is:

Standard attention memory:

$$\Theta(n^2).$$

Linear / kernelized attention

Key difference: you **never materialize** the $n \times n$ attention matrix.

You keep:

1. $\Phi(Q), \Phi(K)$: $\Theta(nm)$ each $\rightarrow \Theta(n)$ in n .
2. Summaries:
 - $S = \Phi(K)^\top V \in \mathbb{R}^{m \times d_v} \rightarrow \Theta(1)$ in n ,
 - $\mathbf{z} = \Phi(K)^\top \mathbf{1} \in \mathbb{R}^m \rightarrow \Theta(1)$ in n .

So peak memory (ignoring implementation details) is:

Linear attention memory:

$$\Theta(n).$$

In streaming / recurrent-style implementations, you can even:

- update S and $\mathbf{z}$ **online** as new tokens arrive,
- so the working memory does **not** grow with n at all (aside from storing past activations for backprop).

3. When is linear attention actually advantageous?

Putting the above together:

Clear wins for linear attention

1. Very long sequences

- When n is large enough that $O(n^2)$ is the dominant bottleneck (both time and memory), linear attention's $\Theta(n)$ scaling becomes a big win.
- Examples: long-context language models, video, audio, genomics, log / event streams.

2. Strict memory constraints (edge / GPU-limited)

- If your hardware cannot store an $n \times n$ attention matrix (or you want bigger batch size / model size within a fixed GPU),
- $O(n)$ memory lets you handle much longer contexts or larger batches.

3. Online / streaming inference

- Because the key-value summaries S , $\mathbf{z}$ can be updated incrementally,
- you can process tokens one-by-one without revisiting the whole past sequence.
- This matches scenarios where the model reads a stream and must keep a compressed state.

Trade-offs / limitations (why we don't always use it)

In homework-style terms, you'd phrase it as:

Linear attention trades exact softmax behavior for better asymptotic scaling.

Concretely:

1. Approximation quality depends on the feature map ϕ

- You're no longer using the exact softmax kernel; you're using a kernel approximation.
- Some choices of ϕ approximate softmax well; others behave quite differently.
- This can affect convergence, stability, and modeling power.

2. Constants and overhead

- Even though asymptotically $\Theta(n)$,
- the constant factors (feature dimension m , extra projections) can make it **slower on short or moderate sequences**.
- So for many typical NLP settings (e.g., $n \leq 1-2k$), vanilla attention is still very competitive.

3. Expressivity and compatibility

- Many empirical tricks for softmax attention (like certain normalization patterns, masking behaviors, etc.) may not transfer directly to a kernelized form, or require extra care.

4. Summary table (single-head, fixed dimensions)

Using the course's simplification (treat d_k, d_v, m as constants):

Mechanism	Time in n	Memory in n	Best use cases
Standard softmax attn	$\Theta(n^2)$	$\Theta(n^2)$	Typical sequence lengths; when exact softmax behavior and mature implementations are desired
Linear / kernelized	$\Theta(n)$	$\Theta(n)$	Very long sequences, memory-limited hardware, online/streaming setups; willing to accept approximation and tweak feature map

If you want, next we can write this comparison exactly in the notation of your “computational complexity of attention” homework question (with h heads and hidden size d), so you can drop it straight into your notes or solutions.

